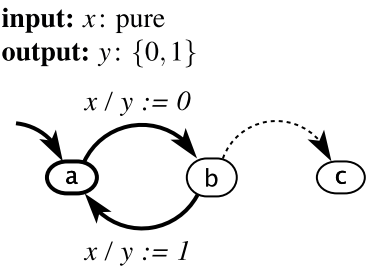
Exercises

1. For each of the following questions, give a short answer and justification.

   (a) TRUE or FALSE: If $\mathbf{GF}p$ holds for a state machine A, then so does $\mathbf{FG}p$.

   (b) TRUE or FALSE: $\mathbf{G}(\mathbf{G}p)$ holds for a trace if and only if $\mathbf{G}p$ holds.

2. Consider the following state machine:



   (Recall that the dashed line represents a default transition.) For each of the following LTL formulas, determine whether it is true or false, and if it is false, give a counterexample:

   (a) $x \implies \mathbf{F}\mathsf{b}$

   (b) $\mathbf{G}(x \implies \mathbf{F}(y = 1))$

   (c) $(\mathbf{G}x) \implies \mathbf{F}(y = 1)$

   (d) $(\mathbf{G}x) \implies \mathbf{GF}(y = 1)$

   (e) $\mathbf{G}((\mathsf{b} \wedge \neg x) \implies \mathbf{FG}\mathsf{c})$

   (f) $\mathbf{G}((\mathsf{b} \wedge \neg x) \implies \mathbf{G}\mathsf{c})$

   (g) $(\mathbf{GF}\neg x) \implies \mathbf{FG}\mathsf{c}$

3. Consider the synchronous feedback composition studied in Exercise 6 of Chapter 6. Determine whether the following statement is true or false:

   The following temporal logic formula is satisfied by the sequence w for every possible behavior of the composition and is not satisfied by any sequence that is not a behavior of the composition:

$$(\mathbf{G}w) \vee (w\mathbf{U}(\mathbf{G}\neg w)) \; .$$

Justify your answer. If you decide it is false, then provide a temporal logic formula for which the assertion is true.

4. This problem is concerned with specifying in linear temporal logic tasks to be performed by a robot. Suppose the robot must visit a set of n locations $l_1, l_2, \ldots, l_n$. Let p_i be an atomic formula that is *true* if and only if the robot visits location l_i.

   Give LTL formulas specifying the following tasks:

   (a) The robot must eventually visit at least one of the n locations.

   (b) The robot must eventually visit all n locations, but in any order.

   (c) The robot must eventually visit all n locations, in the order $l_1, l_2, \ldots, l_n$.

5. Consider a system M modeled by the hierarchical state machine of Figure 13.2, which models an interrupt-driven program. M has two modes: Inactive, in which the main program executes, and Active, in which the interrupt service routine (ISR) executes. The main program and ISR read and update a common variable *timerCount*.

   Answer the following questions:

   (a) Specify the following property ϕ in linear temporal logic, choosing suitable atomic propositions:

   ϕ: The main program eventually reaches program location C.

   (b) Does M satisfy the above LTL property? Justify your answer by constructing the product FSM. If M does not satisfy the property, under what conditions would it do so? Assume that the environment of M can assert the interrupt at any time.

6. Express the postcondition of Example 13.3 as an LTL formula. State your assumptions clearly.

7. Consider the program fragment shown in Figure 11.6, which provides procedures for threads to communicate asynchronously by sending messages to one another. Please answer the following questions about this code. Assume the code is running on a single processor (not a multicore machine). You may also assume that only the code shown accesses the static variables that are shown.

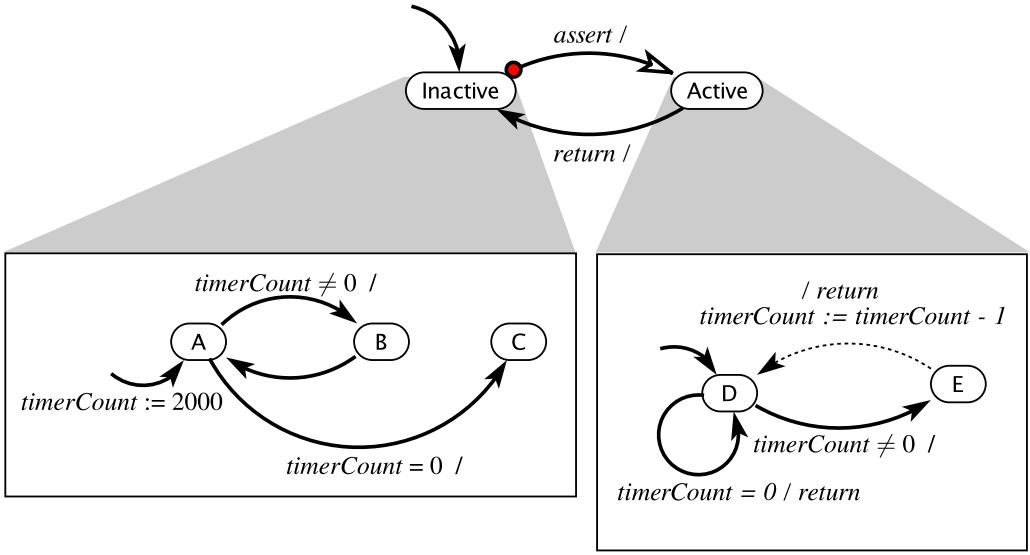


Figure 13.2: Hierarchical state machine modeling a program and its interrupt service routine.

(a) Let s be an atomic proposition asserting that `send` releases the mutex (i.e. executes line 24). Let g be an atomic proposition asserting that `get` releases the mutex (i.e. executes line 38). Write an LTL formula asserting that g cannot occur before s in an execution of the program. Does this formula hold for the first execution of any program that uses these procedures?

(b) Suppose that a program that uses the `send` and `get` procedures in Figure 11.6 is aborted at an arbitrary point in its execution and then restarted at the beginning. In the new execution, it is possible for a call to `get` to return before any call to `send` has been made. Describe how this could come about. What value will `get` return?

(c) Suppose again that a program that uses the `send` and `get` procedures above is aborted at an arbitrary point in its execution and then restarted at the beginning. In the new execution, is it possible for deadlock to occur, where neither

a call to `get` nor a call to `send` can return? If so, describe how this could come about and suggest a fix. If not, give an argument.

Exercises

1. In Figure 14.6 are four pairs of actors. For each pair, determine whether
   - A and B are type equivalent,
   - A is a type refinement of B,
   - B is a type refinement of A, or
   - none of the above.

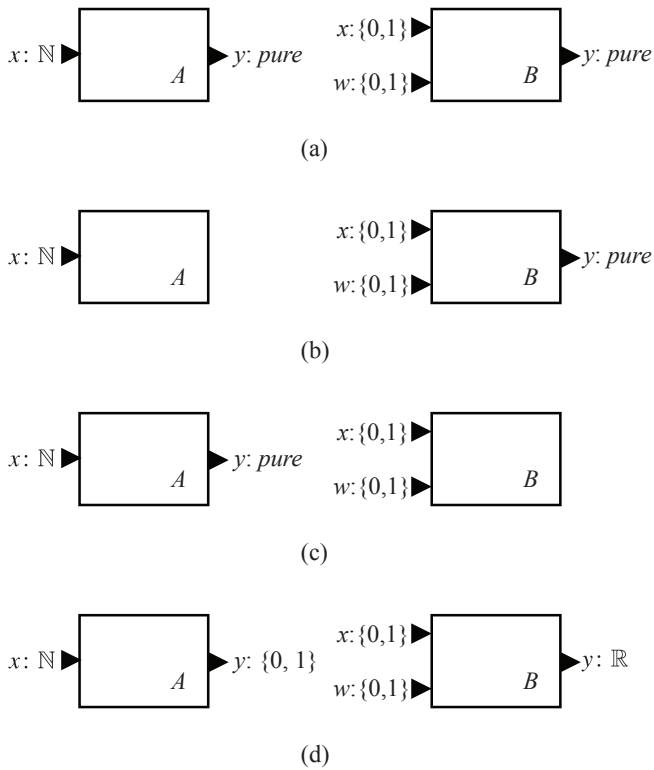


Figure 14.6: Four pairs of actors whose type refinement relationships are explored in Exercise 1.

2. In the box on page 380, a state machine M is given that accepts finite inputs x of the form (1), $(1,0,1)$, $(1,0,1,0,1)$, etc.

   (a) Write a regular expression that describes these inputs. You may ignore stuttering reactions.

   (b) Describe the output sequences in $L_a(M)$ in words, and give a regular expression for those output sequences. You may again ignore stuttering reactions.

   (c) Create a state machine that accepts *output* sequences of the form (1), $(1,0,1)$, $(1,0,1,0,1)$, etc. (see box on page 380). Assume the input x is pure and that whenever the input is present, a present output is produced. Give a deterministic solution if there is one, or explain why there is no deterministic solution. What *input* sequences does your machine accept?

3. The state machine in Figure 14.7 has the property that it outputs at least one 1 between any two 0's. Construct a two-state nondeterministic state machine that simulates this one and preserves that property. Give the simulation relation. Are the machines bisimilar?

4. Consider the FSM in Figure 14.8, which recognizes an input code. The state machine in Figure 14.9 also recognizes the same code, but has more states than the one in Figure 14.8. Show that it is equivalent by giving a bisimulation relation with the machine in Figure 14.8.

5. Consider the state machine in Figure 14.10. Find a bisimilar state machine with only two states, and give the bisimulation relation.

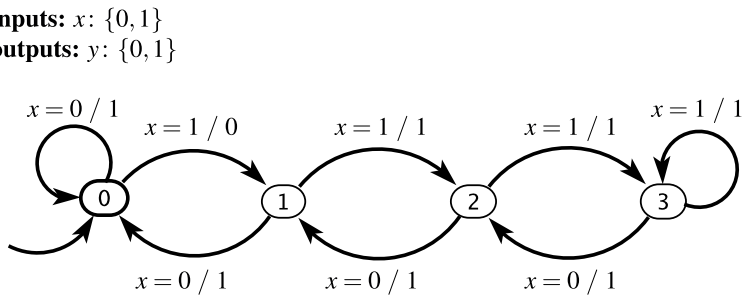


Figure 14.7: Machine that outputs at least one 1 between any two 0's.

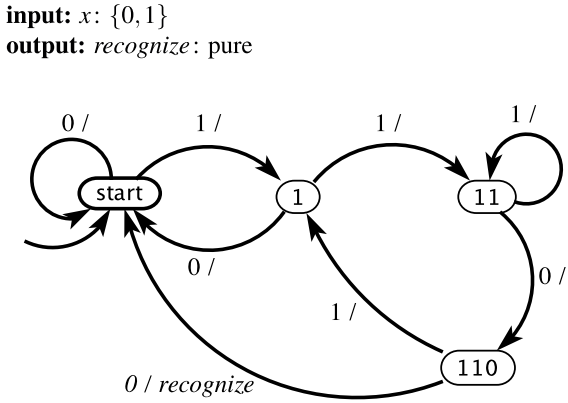


Figure 14.8: A machine that implements a code recognizer. It outputs *recognize* at the end of every input subsequence 1100; otherwise it outputs *absent*.

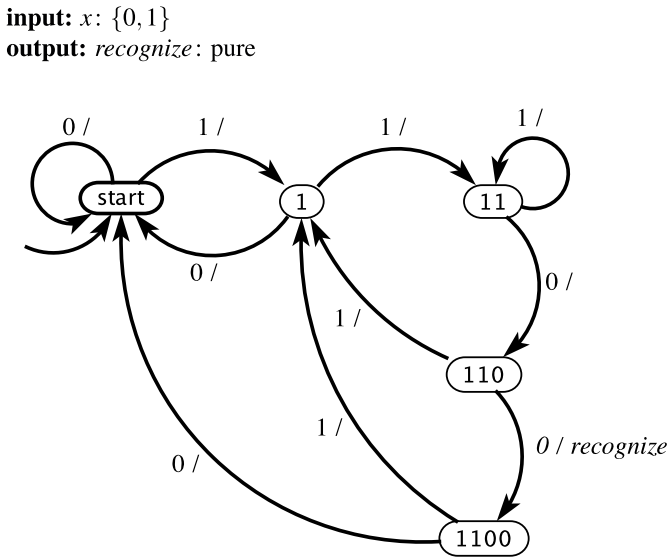


Figure 14.9: A machine that implements a recognizer for the same code as in Figure 14.8, but has more states.

6. You are told that state machine A has one input x, and one output y, both with type $\{1, 2\}$, and that it has states $\{a, b, c, d\}$. You are told nothing further. Do you have enough information to construct a state machine B that simulates A? If so, give such a state machine, and the simulation relation.

7. Consider a state machine with a pure input x, and output y of type $\{0, 1\}$. Assume the states are

$$States = \{a, b, c, d, e, f\},$$

and the initial state is a. The *update* function is given by the following table (ignoring stuttering):

(*currentState*, *input*)	(*nextState*, *output*)
(a, x)	$(b, 1)$
(b, x)	$(c, 0)$
(c, x)	$(d, 0)$
(d, x)	$(e, 1)$
(e, x)	$(f, 0)$
(f, x)	$(a, 0)$

   (a) Draw the state transition diagram for this machine.

   (b) Ignoring stuttering, give all possible behaviors for this machine.

   (c) Find a state machine with three states that is bisimilar to this one. Draw that state machine, and give the bisimulation relation.

8. For each of the following questions, give a short answer and justification.

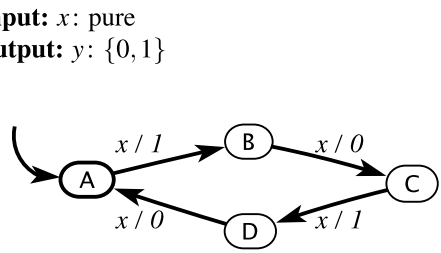


Figure 14.10: A machine that has more states than it needs.

(a) TRUE or FALSE: Consider a state machine A that has one input x, and one output y, both with type $\{1, 2\}$ and a single state s, with two self loops labeled $true/1$ and $true/2$. Then for any state machine B which has exactly the same inputs and outputs (along with types), A simulates B.

(b) TRUE or FALSE: Suppose that f is an arbitrary LTL formula that holds for state machine A, and that A simulates another state machine B. Then we can safely assert that f holds for B.

(c) TRUE or FALSE: Suppose that A and B are two type-equivalent state machines, and that f is an LTL formula where the atomic propositions refer only to the inputs and outputs of A and B, not to their states. If the LTL formula f holds for state machine A, and A simulates state machine B, then f holds for B.